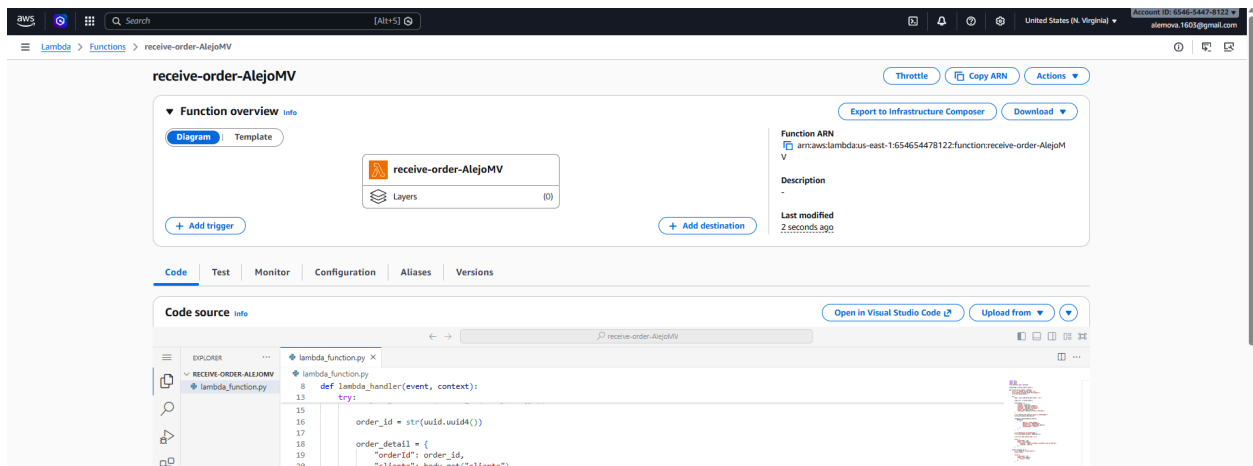


Evidencia clase API gateway y stepfunctions 09/04/2026

1. Creación Lambdas

1. receive-order-AlejoMV



Recibe body desde API gateway

```
import json
import boto3
import uuid
from datetime import datetime

eventbridge = boto3.client('events')

def lambda_handler(event, context):
    print("=== INICIO receive-order ===")
    print("Evento recibido desde API Gateway:")
    print(json.dumps(event))

    try:
```

```

body = json.loads(event.get('body', '{}'))

order_id = str(uuid.uuid4())

order_detail = {
    "orderId": order_id,
    "cliente": body.get("cliente"),
    "producto": body.get("producto"),
    "cantidad": body.get("cantidad"),
    "total": body.get("total"),
    "timestamp": datetime.utcnow().isoformat()
}

print("Detalle del pedido a enviar a EventBridge:")
print(json.dumps(order_detail))

response = eventbridge.put_events(
    Entries=[
        {
            "Source": "demo.pedidos",
            "DetailType": "NuevoPedido",
            "Detail": json.dumps(order_detail),
            "EventBusName": "default"
        }
    ]
)

print("Respuesta de EventBridge:")
print(json.dumps(response, default=str))

print("=== FIN receive-order ===")

return {
    "statusCode": 200,
    "body": json.dumps({
        "mensaje": "Pedido recibido y enviado al bus

```

```

de eventos",
        "orderId": order_id
    })
}

except Exception as e:
    print("ERROR en receive-order:")
    print(str(e))

    return {
        "statusCode": 500,
        "body": json.dumps({
            "error": str(e)
        })
    }

```

Le damos los permisos necesarios

Modify permissions in `PermiteEstablecerEventos` [Info](#)

Add permissions by selecting services, actions, resources, and conditions. Build permission statements using the JSON editor.

The screenshot shows the AWS IAM Policy Editor interface. At the top, there are tabs for 'Visual', 'JSON' (which is selected), and 'Actions'. Below the tabs, the 'Policy editor' title is visible. On the left, a JSON editor shows a statement with the following content:

```

1  [
2    {
3      "Version": "2012-10-17",
4      "Statement": [
5        {
6          "Effect": "Allow",
7          "Action": "events:PutEvents",
8          "Resource": "*"
9        }
10   ]

```

On the right side of the interface, there is a section titled 'Edit statement' with a plus icon. Below this, there is a button labeled 'Select a statement'.

2. Secundarias del step-functions

1. validate-order-AlejoMV

```

import json

def lambda_handler(event, context):
    print("=== INICIO validate-order ===")
    print("Evento recibido:")
    print(json.dumps(event))

    cliente = event.get("detail", {}).get("cliente")
    producto = event.get("detail", {}).get("producto")
    cantidad = event.get("detail", {}).get("cantidad", 0)
    total = event.get("detail", {}).get("total", 0)

    is_valid = all([
        cliente is not None and cliente != "",
        producto is not None and producto != "",
        cantidad > 0,
        total > 0
    ])

    result = event.get("detail", {}).copy()
    result["validationStatus"] = "VALID" if is_valid else "INVALID"

    print("Resultado de validación:")
    print(json.dumps(result))
    print("=== FIN validate-order ===")

    return result

```

2. check-inventory-AlejoMV

```

import json

def lambda_handler(event, context):

```

```

print("=== INICIO check-inventory ===")
print("Evento recibido:")
print(json.dumps(event))

cantidad = event.get("cantidad", 0)

if cantidad <= 5:
    event["inventoryStatus"] = "AVAILABLE"
else:
    event["inventoryStatus"] = "OUT_OF_STOCK"

print("Resultado inventario:")
print(json.dumps(event))
print("=== FIN check-inventory ===")

return event

```

3. process-payment-AlejoMV

```

import json

def lambda_handler(event, context):
    print("=== INICIO process-payment ===")
    print("Evento recibido:")
    print(json.dumps(event))

    total = event.get("total", 0)

    if total <= 1000:
        event["paymentStatus"] = "APPROVED"
    else:
        event["paymentStatus"] = "REJECTED"

    print("Resultado pago:")
    print(json.dumps(event))

```

```
print("=== FIN process-payment ===")

return event
```

4. confirm-order-AlejoMV

```
import json

def lambda_handler(event, context):
    print("=== INICIO confirm-order ===")
    print("Evento recibido:")
    print(json.dumps(event))

    event["orderStatus"] = "CONFIRMED"

    print("Pedido confirmado:")
    print(json.dumps(event))
    print("=== FIN confirm-order ===")

    return event
```

5. register-audit-AlejoMV

```
import json

def lambda_handler(event, context):
    print("=== INICIO register-audit ===")
    print("Evento recibido para auditoría:")
    print(json.dumps(event))

    result = {
        "mensaje": "Proceso finalizado",
        "orderId": event.get("orderId"),
        "cliente": event.get("cliente"),
        "producto": event.get("producto"),
```

```

        "cantidad": event.get("cantidad"),
        "total": event.get("total"),
        "orderStatus": event.get("orderStatus", "FAILED"),
        "inventoryStatus": event.get("inventoryStatus"),
        "paymentStatus": event.get("paymentStatus")
    }

    print("Auditoría final:")
    print(json.dumps(result))
    print("=== FIN register-audit ===")

    return result

```

2. creación StepFunction

Reemplazar los ARN

```

{
    "Comment": "Procesamiento de pedidos con Step Functions",
    "StartAt": "ValidateOrder",
    "States": {
        "ValidateOrder": {
            "Type": "Task",
            "Resource": "arn:aws:lambda:us-east-1:654654478122:func
tion:validate-order-AlejoMV",
            "OutputPath": "$.Payload",
            "Parameters": {
                "FunctionName": "ARN_VALIDATE_ORDER",
                "Payload.$": "$"
            },
            "Next": "ValidationChoice"
        },
        "ValidationChoice": {
            "Type": "Choice",
            "Choices": [

```

```

        {
            "Variable": "$.validationStatus",
            "StringEquals": "VALID",
            "Next": "CheckInventory"
        }
    ],
    "Default": "InvalidOrderFail"
},
"CheckInventory": {
    "Type": "Task",
    "Resource": "arn:aws:lambda:us-east-1:654654478122:func
tion:check-inventory-AlejoMV",
    "OutputPath": "$.Payload",
    "Parameters": {
        "FunctionName": "ARN_CHECK_INVENTORY",
        "Payload.$": "$"
    },
    "Next": "InventoryChoice"
},
"InventoryChoice": {
    "Type": "Choice",
    "Choices": [
        {
            "Variable": "$.inventoryStatus",
            "StringEquals": "AVAILABLE",
            "Next": "ProcessPayment"
        }
    ],
    "Default": "OutOfStockFail"
},
"ProcessPayment": {
    "Type": "Task",
    "Resource": "arn:aws:lambda:us-east-1:654654478122:func
tion:process-payment-AlejoMV",
    "OutputPath": "$.Payload",
    "Parameters": {

```



```

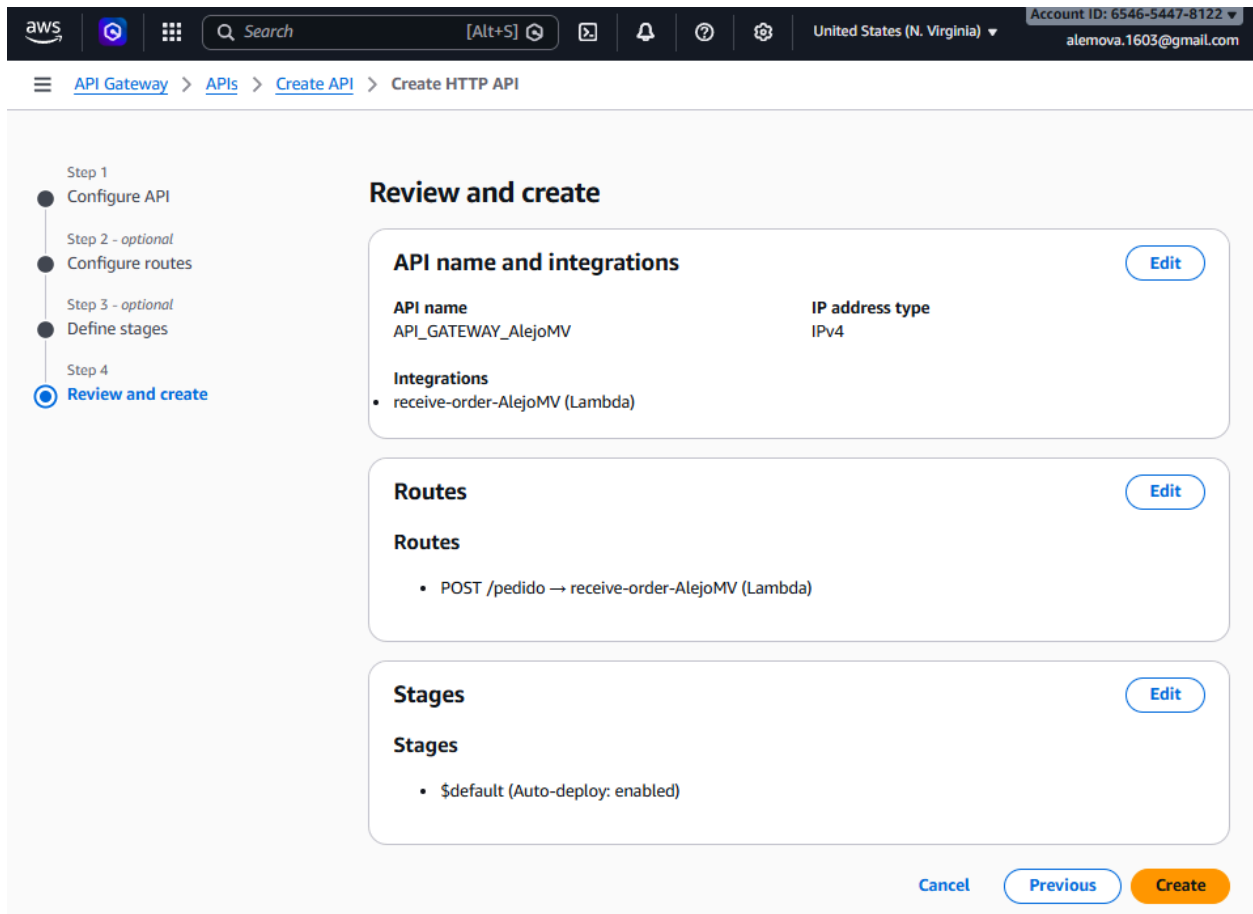
        "FunctionName": "ARN_PROCESS_PAYMENT",
        "Payload.$": "$"
    },
    "Next": "PaymentChoice"
},
"PaymentChoice": {
    "Type": "Choice",
    "Choices": [
        {
            "Variable": "$.paymentStatus",
            "StringEquals": "APPROVED",
            "Next": "WaitForConfirmation"
        }
    ],
    "Default": "PaymentRejectedFail"
},
"WaitForConfirmation": {
    "Type": "Wait",
    "Seconds": 5,
    "Next": "ConfirmOrder"
},
"ConfirmOrder": {
    "Type": "Task",
    "Resource": "arn:aws:lambda:us-east-1:654654478122:func
tion:confirm-order-AlejoMV",
    "OutputPath": "$.Payload",
    "Parameters": {
        "FunctionName": "ARN_CONFIRM_ORDER",
        "Payload.$": "$"
    },
    "Next": "RegisterAudit"
},
"RegisterAudit": {
    "Type": "Task",
    "Resource": "arn:aws:lambda:us-east-1:654654478122:func
tion:register-audit-AlejoMV",

```

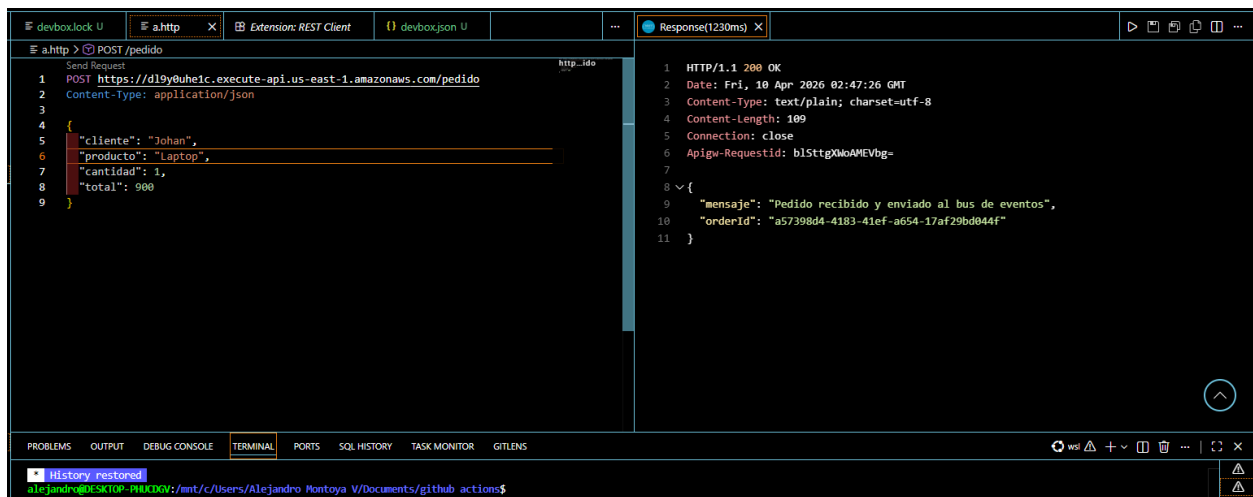
```

    "OutputPath": "$.Payload",
    "Parameters": {
      "FunctionName": "ARN_REGISTER_AUDIT",
      "Payload.$": "$"
    },
    "Next": "SuccessState"
  },
  "SuccessState": {
    "Type": "Succeed"
  },
  "InvalidOrderFail": {
    "Type": "Fail",
    "Error": "InvalidOrder",
    "Cause": "El pedido no es válido"
  },
  "OutOfStockFail": {
    "Type": "Fail",
    "Error": "OutOfStock",
    "Cause": "No hay inventario disponible"
  },
  "PaymentRejectedFail": {
    "Type": "Fail",
    "Error": "PaymentRejected",
    "Cause": "El pago fue rechazado"
  }
}
}

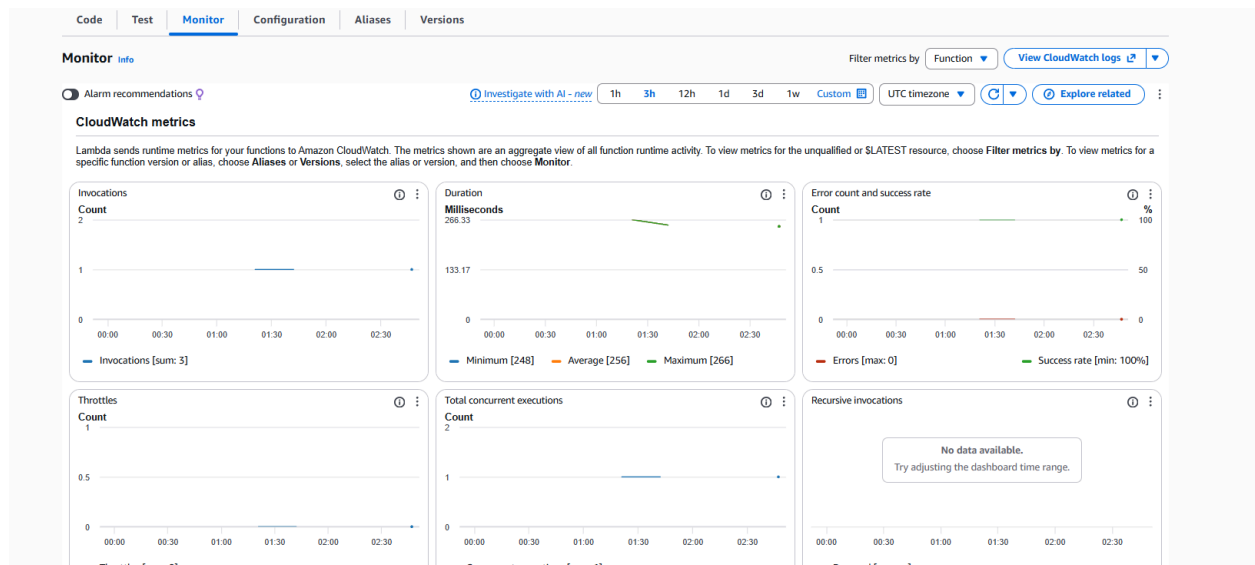
```

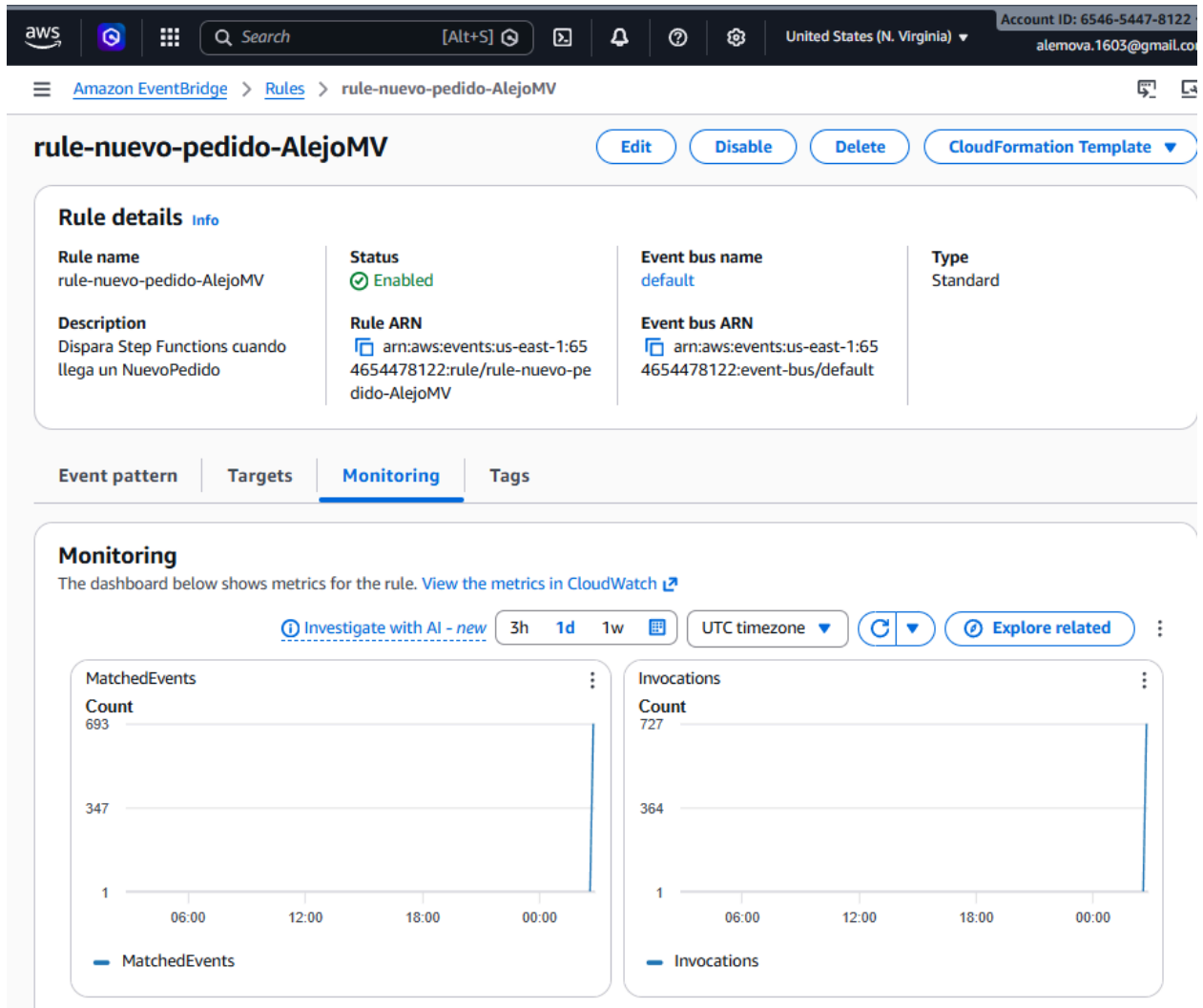
5. Prueba de ejecución



Vemos que se activo lambda



Event bridge



Step functions